

Exact acceleration of the supplied unknot recognizer

Sparse cancellation, bounded Jones obstructions,
and visible connected-sum factorization

Implementation and analysis report

18 September 2026 — Version 0.2.0

Abstract

We modify the actual `fast/` implementation in the submitted `Knots.zip`, retaining its exact characteristic-two Khovanov backend. The delivered software introduces fill-sensitive pivot scheduling, algebraic memoization, a self-inverse-unit shortcut, a bounded scalar Jones state sum, and diagrammatic connected-sum factorization. It also accelerates two preprocessing routines and repairs crossing-sign handling for general PD representatives. On same-host, fresh-process benchmarks, default recognition of the Conway and Kinoshita–Terasaka fixtures is approximately 39 and 44 times faster, respectively. A difficult 36-crossing raw scan finishes in 1.285 seconds, whereas the unchanged baseline reaches a 20-second cap. An ablation using the original uncached algebra confirms its reduced rank of 2949.

No unrestricted quasi-polynomial bound is established. We prove instead the bound $\text{poly}(n) + n2^{O(m)}$, where m is the largest visible summand left by the implemented factorization. This gives $n^{O(\log n)}$ on the class $m = O((\log n)^2)$, and polynomial time on visible connected sums of bounded-size diagrams. For a concrete family, it avoids the baseline’s exponentially large homology basis even when both polynomial-invariant filters are disabled. The report separates these theorems from empirical speedups, describes the exactness and resource-limit contracts, and includes reproducible code, tests, inputs, and raw measurements.

Contents

1	Scope, baseline, and the requested target	3
2	Representation and trusted mathematical ingredients	3
2.1	The scanning category	3
2.2	Why unreduced rank two detects the unknot	4
3	Accelerating the categorical scanner	4
3.1	Fill-sensitive cancellation	4
3.2	Units are self-inverse in the implemented algebra	5
3.3	Memoization without mutable-alias errors	6
3.4	Exactly the same greedy order, faster	6
4	A bounded, exact Jones obstruction	6
4.1	State sum and finite-ring specialization	7
4.2	Frontier dynamic programming	7
4.3	Orientation repair needed for exact normalization	8
5	Visible connected sums: avoiding a tensor-product explosion	8
5.1	Finding and replaying a cut	8
5.2	Reduced ranks multiply	9
5.3	An exponential separation from the supplied algorithm	10
6	The complete recognition pipeline	10
6.1	The linear descending test	11
7	Complexity: what is proved and what is not	11
7.1	An unrestricted exponential upper bound	11
7.2	The visible-factor parameter	12
7.3	Why a thin boundary does not prove fast Khovanov rank	13
7.4	Attempting to reach the hierarchy target	13
8	Reproducible experiments	14
8.1	Protocol and machine	14
8.2	Default end-to-end algorithmic timings	14
8.3	Raw scanning, without preprocessing or invariant shortcuts	15
8.4	Ablating pivot policy and algebra caching	15
8.5	Factorization and ordering experiments	16
9	Validation and limitations of the evidence	16
10	Interfaces, evidence, resource budgets, and reproducibility	17
10.1	Commands and library entry points	17
10.2	Evidence is contextual, not a universal certificate	18
10.3	Limits are explicit but cooperative	18
10.4	Rebuilding the measurements and article	18
11	Conclusion and the next concrete bottleneck	19

1 Scope, baseline, and the requested target

The comparison target is the seven-module Python package under `fast/fastunknot/` in the uploaded archive. It is preserved, byte for byte, as `baseline/fastunknot/` in the deliverable. The modified package is `fastunknot/`. No alternative implementation from the archive’s six older attempts is substituted as the baseline. Source provenance, including the identity of the submitted ZIP, is recorded in `PROVENANCE.md`.

The baseline validates a classical, single-component planar diagram, performs crossing-decreasing Reidemeister I and II moves, tests for a descending basepoint, computes an Alexander polynomial, and finally computes Khovanov rank over $\mathbb{F}_2$. Its last stage is a local scanning implementation in the style of Bar-Natan [1], not the naive full cube of resolutions. Our changes build on that existing work; neither scanning nor the underlying knot invariants are claimed as new discoveries.

What the supplied Lackenby materials say. The submitted February 2021 talk states the $n^{O(\log n)}$ target on physical page 13 and outlines accelerated hierarchy operations on page 109 [4]. The related July 2026 paper proves a hierarchy-based recognition algorithm. Section 9 gives an iteration estimate in terms of hierarchy length and surface complexity, rather than a crossing-number running-time bound [6]. Section 7.4 explains precisely why our implementation has not converted those operations into an unrestricted quasi-polynomial algorithm.

There are distinct public slide versions: the author’s March 2021 Oxford variant states $2^{O((\log n)^3)}$, whereas the February file actually supplied here states $n^{O(\log n)}$. We use the latter as the requested target rather than silently identifying these different bounds [4, 5].

The delivered result. The software is executable using Python 3.10 or later and its standard library. It has no missing hierarchy subroutine hidden behind an oracle. With unlimited resources it retains an exact complete recognition route; configured limits can instead return `UNKNOWN`. Its general worst-case bound remains exponential. The strongest new asymptotic statement is a visible-factor parameter bound, and the strongest observed backend improvement comes from changing the cancellation order, not merely adding a faster knot invariant.

2 Representation and trusted mathematical ingredients

A PD row (a, b, c, d) lists the four ports of a crossing counterclockwise. Ports 0 and 2 belong to the underpassing strand, and ports 1 and 3 to the overpassing strand. Each edge label occurs twice. The public constructors normalize labels, check the opposite-port traversal has one component, and check the rotation system is spherical. The empty diagram denotes one unknot, not an empty link. These conventions are inherited and matter for every subsequent exactness statement.

2.1 The scanning category

At a partial stage, a crossing-free object is a matching M of the current boundary endpoints, together with a homological degree h . Closed circles are immediately delooped into two copies, labeled 1 and x . A morphism between matchings is a characteristic-two sum of dotted surfaces, represented in a basis indexed by dot subsets of the circles formed by joining the two matchings.

The relevant Frobenius algebra is

$$A = \mathbb{F}_2[x]/(x^2), \quad \Delta(1) = 1 \otimes x + x \otimes 1, \quad \Delta(x) = x \otimes x, \quad \epsilon(1) = 0, \quad \epsilon(x) = 1.$$

The baseline already implements tensoring with a crossing, delooping, composition, and cancellation of invertible differential entries. These are local versions of the formal-complex operations of [1]. Our optimized scanner retains the same mathematical computation.

At the end of a closed diagram, the matching is empty. All nonzero characteristic-two scalar differential entries are units, so cancellation leaves a zero differential. The remaining number of objects is the total unreduced Khovanov rank. Homological degrees retained by this implementation are *raw cube degrees*; writhe shifts and quantum gradings are not reported. Total ranks suffice for recognition.

2.2 Why unreduced rank two detects the unknot

We use the established theorem that reduced Khovanov homology of rank one detects the unknot [2]. Its application to the particular field and unreduced implementation deserves explanation.

Lemma 2.1. *For a knot diagram, after forgetting the quantum grading,*

$$\dim_{\mathbb{F}_2} \text{Kh}^h(K; \mathbb{F}_2) = 2 \dim_{\mathbb{F}_2} \widetilde{\text{Kh}}^h(K; \mathbb{F}_2)$$

in each homological degree.

Proof. Let X multiply the label on a marked circle by x . Let ν be the sum of the operations that change one circle label x to 1 and annihilate label 1. The formulas for m and Δ show that ν commutes with the characteristic-two differential. They also give

$$X^2 = \nu^2 = 0, \quad X\nu + \nu X = 1.$$

Consequently $X\nu$ and νX are complementary chain projections. Their images are isomorphic by X and ν , and the first is the reduced subcomplex $\text{im } X$. Neither operator changes homological degree. Taking homology gives the assertion. $\square$

If the reduced $\mathbb{F}_2$ rank is one, the universal coefficient theorem implies that the reduced rational rank is at most one. It is at least one: the normalized reduced Euler characteristic has evaluation one at the unit parameter, so the rational complex cannot have zero homology. The rank-one unknot-detection theorem therefore applies. Conversely the reduced homology of the unknot has rank one. Thus the backend's unreduced rank test is exactly a rank-two test, not a heuristic comparison with a familiar knot table. The software does not reprove the deep detection theorem; it relies on it just as the baseline does.

3 Accelerating the categorical scanner

3.1 Fill-sensitive cancellation

Write a differential block around an invertible entry $\phi : B \rightarrow C$ as

$$d = \begin{pmatrix} \phi & \delta \\ \gamma & \varepsilon \end{pmatrix} : B \oplus U \longrightarrow C \oplus V.$$

Removing the contractible pair B, C replaces the remaining block by

$$\varepsilon' = \varepsilon + \gamma\phi^{-1}\delta. \quad (1)$$

The plus sign is correct over $\mathbb{F}_2$. This cancellation is a chain-homotopy equivalence; it is the categorical Gaussian elimination used in the baseline and in [1]. The algebraic validity does not depend on which available invertible entry is chosen.

For a candidate arrow $b \rightarrow c$, let r_c be the number of arrows entering c and s_b the number leaving b . Cancelling it may update at most

$$(r_c - 1)(s_b - 1) \quad (2)$$

other entries. The old implementation uses a stack of candidates. The new one uses a minimum heap keyed by this local fill estimate, then by a serial number for deterministic ties. A candidate is rechecked for existence and invertibility before cancellation. Its numerical score is the score at insertion, not a continuously maintained exact minimum.

Proposition 3.1. *Stale scores in this queue cannot invalidate the homology computation.*

Proof. A score only selects which candidate to inspect. The current morphism and its current source and target matchings are checked again. Every performed operation is therefore a valid instance of (1). Removed or no longer invertible candidates are discarded. Any entry that changes to a nonzero unit is reinserted. The process terminates because each cancellation removes two objects. Selection by a stale score changes performance, not the resulting homotopy type. $\square$

This is a heuristic for sparsity, not a theorem that the number of intermediate objects is polynomial. It is nevertheless the decisive change on the 36-crossing fixture. A separate ablation changes only this cancellation loop while continuing to use the baseline's uncached algebra.

3.2 Units are self-inverse in the implemented algebra

For a matching with k arcs, its endomorphism ring in this normal form is

$$R_k = \mathbb{F}_2[x_1, \dots, x_k] / (x_1^2, \dots, x_k^2). \quad (3)$$

The basis monomials are square-free dot subsets. The identity is the empty subset, and an endomorphism is a unit exactly when the identity coefficient is one.

Lemma 3.2. *Every unit of R_k is its own inverse.*

Proof. Write a unit as $1 + u$, where u has zero constant coefficient. In a commutative ring of characteristic two the square of a sum is the sum of the squares. Every nonconstant square-free monomial has square zero in (3). Thus $u^2 = 0$ and $(1 + u)^2 = 1$. $\square$

The old inverse routine computes a nilpotent expansion using general surface composition. The replacement returns a copy of the input morphism. The restriction to *endomorphisms with identical source and target matching* is essential: a term that looks empty in a general Hom space is not by itself an identity between different objects. The pivot and identity fast paths both check this restriction. Exhaustive tests enumerate all 128 units for a three-arc matching and square each using the original composition routine.

3.3 Memoization without mutable-alias errors

The same small matching and the same local morphism can occur on many copies of a chain object. Recomputing their topological gluing for each copy wastes work unrelated to the number of genuinely different local maps.

The modified implementation has three kinds of reuse. First, composition returns zero immediately on a zero input and copies the other argument for a valid identity input. Otherwise its key contains the three matchings and *frozen* copies of the two morphisms. The cached value is frozen, and the public result is copied back into an owned mutable set. Second, geometric circle and smoothing-gluing caches are bounded to 8192 entries each; the composition cache is bounded to 32768. Third, each crossing has a temporary cache of the delooped entries for a given pair of matchings, morphism, and pair of smoothing indices. Boundary and crossing data are constant within that temporary cache, which is discarded at the end of the crossing.

The entry maps shared between chain arrows are read-only by convention. When cancellation updates a differential entry it first copies it and then performs symmetric difference. This prevents one arrow's cancellation from mutating another arrow or a cached result. The cache limits count entries, not bytes; a single morphism can still be large. `clear_caches()` permits explicit release between applications.

On the recorded difficult scan, the crossing-map cache has 195674 hits and 3154 misses. The optimized run performs 16884 cancellations and records 168172 calls to composition inside cancellation, with a maximum of 17694 objects before elimination. These are measured work counters, not asymptotic bounds.

3.4 Exactly the same greedy order, faster

For an unprocessed crossing, write s for the number of its ports incident to the current boundary, b for the boundary size, and ℓ for its number of repeated edge labels. The baseline's lexicographic score is

$$(s, -b - 4 + 2s + 2\ell, -i),$$

where i is the crossing index. Since b is common to all candidates and s is compared first, this induces exactly the same order as $(s, \ell, -i)$. Only neighbors of a newly scanned crossing can change s . On a four-valent PD graph there are $O(n)$ such updates in total.

Maintaining this score in a versioned heap therefore requires $O(n \log n)$ heap time and $O(n)$ heap records per starting crossing, instead of rescoring $O(n)$ candidates at $O(n)$ stages. The multi-start routine retains the baseline's start selection and boundary-profile tie breaking. Its default samples only a bounded number of starts, hence remains $O(n \log n)$ overall. Explicitly requesting every start instead costs $O(n^2 \log n)$. Tests compare the actual returned permutations with the baseline for every start on 100 generated diagrams.

4 A bounded, exact Jones obstruction

The Jones layer is a separate scalar computation. It does not call the categorical surface-composition or gluing routines and does not compute Khovanov homology. The shared infrastructure consists only of validated diagrams, orientation, and a crossing-order heuristic.

4.1 State sum and finite-ring specialization

Use the bracket convention $\langle \bigcirc \rangle = 1$, smoothings $(a, b)(c, d)$ and $(a, d)(b, c)$ with respective weights A and A^{-1} , and

$$\delta = -A^2 - A^{-2}.$$

For a nonempty diagram D with n crossings, a smoothing state s with $|s|$ second smoothings and $c(s)$ circles contributes

$$\langle D \rangle = \sum_{s \in \{0,1\}^n} A^{n-2|s|} \delta^{c(s)-1}. \quad (4)$$

The writhe-normalized quantity is

$$J_A(D) = (-A^3)^{-w(D)} \langle D \rangle. \quad (5)$$

It is the usual normalized bracket version of the Jones invariant, with variable choice $t = A^{-4}$ [3]. Our PD and writhe conventions are paired consistently; swapping to a mirror replaces the corresponding variable convention but does not change the unknot criterion.

The implementation evaluates in $\mathbb{Z}/M\mathbb{Z}$. It checks that both A and δ are invertible and uses exact modular inverses. Primality of M is not needed. The default is $M = 1000000007$, $A = 2$.

Proposition 4.1. *A successfully computed residue $J_A(D) \neq 1$ certifies **KNOTTED**. The residue $J_A(D) = 1$ gives no acceptance criterion for **UNKNOT**.*

Proof. For an unknot, the normalized Laurent polynomial (5) is identically one. Specializing it at units in any admissible finite ring still gives one. Contraposition proves the rejection criterion. Distinct Laurent polynomials may agree at the chosen ring element, so equality at one value cannot prove polynomial equality or unknotness. $\square$

This is not a randomized test with a small probability of false rejection. A collision can only make the filter unhelpful. No conjecture about whether the Jones polynomial detects the unknot is used.

4.2 Frontier dynamic programming

After processing some crossings, store a dictionary from boundary matchings to scalar partial weights. Adding a crossing has two transitions per matching. A small disjoint-set computation joins the old matching to the chosen local smoothing. A resulting component has either two new boundary endpoints, which become an arc in the new matching, or no endpoints, which contributes one factor δ . Anything else is rejected as inconsistent geometry. Weights of equal new matchings are summed modulo M .

For convenience, the dynamic program accumulates $\delta^{c(s)}$ rather than $\delta^{c(s)-1}$; it divides by δ once at the end. The zero-crossing unknot is treated separately as value one. Induction on processed crossings proves that the dictionary is exactly the partial sum obtained by aggregating all smoothing choices with the same boundary matching. Final closure then gives (4), and writhe normalization gives (5).

If the boundary has at most w endpoints, the number of arbitrary matching states is at most $(w-1)!! = 2^{O(w \log w)}$. We do *not* substitute a Catalan bound without proving that the boundary lies on one disk in the appropriate cyclic order: a general scan need not have that form. There are also

at most 2^n partial smoothing states before aggregation. This scalar algorithm can be exponential and is not the complete recognizer.

The default filter is bounded by 100000 stored states and 200000 transitions. Reaching either work cap skips the filter and continues with Alexander and then the exact scanner. With a fixed-size modulus and the default work cap, its overhead is polynomial in input size. A global time deadline has a different meaning: it terminates the recognition attempt with UNKNOWN.

4.3 Orientation repair needed for exact normalization

The baseline's crossing signs depend only on which over-port is incoming, as though the incoming under-port were always slot 0. A legal two-slot rotation of an individual PD row preserves the crossing but can violate that assumption. Let $u \in \{0, 2\}$ and $o \in \{1, 3\}$ be the actual incoming under- and over-port. The repaired local sign is

$$\varepsilon = \begin{cases} +1, & o - u \equiv 3 \pmod{4}, \\ -1, & o - u \equiv 1 \pmod{4}. \end{cases}$$

Simultaneously adding two to both incoming ports leaves it unchanged.

It would be unsafe to change `signs()` alone: Alexander matrix construction previously used fixed under-port positions in conjunction with the old effective sign. It now identifies the actual incoming under-arc and interchanges its under-arc variables when needed. Tests check writhe, Alexander polynomial, and Jones residues under independent two-slot row rotations on 100 generated diagrams, as well as reversal of writhe on mirrors.

5 Visible connected sums: avoiding a tensor-product explosion

The baseline always computes a final homology basis if its Alexander test fails. A small frontier cannot prevent this basis from being enormous. Connected sums expose a way to avoid constructing it while still deciding unknotness, or even computing its rank.

5.1 Finding and replaying a cut

Traverse the oriented knot once and record the crossing index at each visit. The resulting cyclic Gauss word has length $2n$, with each crossing appearing twice. For each possible start, extend a proper cyclic interval and maintain the set of crossing indices seen an odd number of times. An empty odd set means that every crossing touched by that interval has both visits inside it.

Lemma 5.1. *A nonempty proper closed interval of this kind determines a visible connected-sum decomposition of the spherical knot diagram.*

Proof. Let S be the set of its crossings. Along the knot traversal, every edge between visits inside the interval has both endpoints in S , and similarly for the complementary interval. Precisely the incoming and outgoing boundary edges cross from S to its complement. Both induced subgraphs are connected, since each is traversed by its interval. Thus the two boundary edges form a bond of the plane projection graph. Its dual is a simple two-edge cycle, realizable as a Jordan curve avoiding vertices and meeting those two edges once each. This curve is a diagrammatic connected-sum circle. Closing each of the two resulting arcs gives the two summands. $\square$

The implementation closes a side by identifying its two cut labels, retains its PD rows, and runs the usual diagram validator. It repeats this operation on an explicit stack, avoiding recursion-depth limits. A search on a k -crossing diagram costs $O(k^2)$ set operations; at most $n - 1$ cuts give the conservative overall bound $O(n^3)$, before label bit costs. A more specialized linear-time bond algorithm could reduce this overhead, but is not needed for the claimed bound.

Each cut records its current node number, child numbers, the two edge labels, and the chosen crossing subset in that node's normalized PD. The replay routine checks the exact cut edges, reconstructs both closures, and validates the resulting diagrams. These records describe actual visible cuts, not asserted knot names. Failure to find a cut says nothing about topological primality: a composite knot can have a projection without such an exposed cut.

5.2 Reduced ranks multiply

Proposition 5.2. *For the visible connected sum $K \# L$ over $\mathbb{F}_2$, reduced Khovanov complexes have the connected-sum tensor description, and hence*

$$r(K \# L) = r(K)r(L), \quad (6)$$

$$r_h(K \# L) = \sum_{i+j=h} r_i(K)r_j(L), \quad (7)$$

where r is total reduced rank and the r_h use the compatible raw cube homological degrees of the spliced diagrams.

Proof. Mark a point on each spliced edge. Each full cube state is a free A -module through the marked circle. A state of the connected sum consists of one state of each summand, with the marked circles identified; its module is their tensor product over A . The crossing differentials lie in one summand or the other, and this identification respects multiplication and comultiplication. Thus the connected-sum complex is the corresponding tensor product over A .

Reduced complexes may be represented by base change along $A \rightarrow A/(x) = \mathbb{F}_2$; this is isomorphic, up to the irrelevant quantum shift, to taking marked label x . Base change of the displayed tensor product yields the tensor product of the two reduced complexes over $\mathbb{F}_2$. Homology of a tensor product of finite complexes over a field is the tensor product of their homologies. Its dimension and homological grading give (6) and (7). $\square$

Every summand has positive integral reduced rank. Therefore the product is one exactly when every factor is one. Together with the detection theorem, this gives a particularly simple exact decision strategy: accept only when all visible factors are accepted, and reject as soon as one is proved knotted. It does not require a general topological prime-factorization algorithm.

The separate `factorized_khovanov_rank` function computes the product and degree convolution instead of a basis. Lemma 2.1 converts the scanner's per-degree unreduced counts into reduced counts by dividing by two, then doubles the final counts for the output's unreduced convention. An odd intermediate count is treated as an error. Identical normalized PD leaves are memoized; this is an exact equality test, not a knot-isomorphism heuristic. Different presentations of the same knot may be recomputed.

5.3 An exponential separation from the supplied algorithm

Let D be the supplied eleven-crossing Conway fixture and form $D^{\#k}$ by the edge splice in `tools/common.py`. On D , the independent dense reference computation gives reduced rank 33, the exact Alexander routine gives 1, and inspection finds no legal R1 or R2 move. These checks are retained in `results/independent-dense-named.json`.

Proposition 5.3. *On this family, the supplied complete baseline requires $\Omega(33^k)$ stored final chain objects. The new recognition algorithm runs in polynomial time in k , even with both Jones and Alexander filters disabled.*

Proof. The repeated edge splice displays all k factors. It introduces no crossing, no monogon, and no new two-crossing bigon: the two splicing edges have distinct endpoints in each copy, and a newly joined face runs through both sides of the splice. Old faces not incident to the splice are unchanged. Consequently the baseline’s crossing-decreasing reductions do not remove these factors. The connected sum is nontrivial by (6), so it has no descending basepoint. Its Alexander polynomial is the product of k unit polynomials, still one.

Thus the baseline reaches its scanner. The final unreduced homology has rank $2 \cdot 33^k$. That implementation has zero final differential and explicitly retains one object per generator, hence must store at least that many objects before returning. This lower bound is on this supplied implementation, not on every possible algorithm for the decision problem.

The new decomposition finds visible factors of size eleven in polynomial time. It computes the first such factor’s rank and immediately rejects; there is no need to construct any tensor-product basis. The fixed-size factor computation is a constant in the family parameter k . Polynomial decomposition and preprocessing therefore dominate. $\square$

The same argument applies to any fixed reduced diagram of a knot with Alexander polynomial one and reduced rank greater than one, with an exposed splice that preserves the absence of reducing moves. It is a decision-specific improvement: building the whole homology is unnecessary for deciding whether its reduced dimension equals one.

The factored-rank API goes further on the concrete $k = 12$ fixture: it returns

$$r(D^{\#12}) = 33^{12} = 1667889514952984961$$

in about 0.032 seconds in the recorded run. The integer has a small binary encoding, although a vector-space basis with that many elements does not. The homological count array is also compact. This measurement is for the factored-rank algorithm, not for a raw scan of 132 crossings.

6 The complete recognition pipeline

The new default is

validate $\longrightarrow$ R1/R2 reduction $\longrightarrow$ descending test
 $\longrightarrow$ visible factorization $\longrightarrow$ per-factor bounded Jones test
 $\longrightarrow$ per-factor Alexander test $\longrightarrow$ accelerated exact scan.

Each factor is simplified and tested for descent again. Small factors are processed first. Repeated identical PD leaves share their computed decision. A knotted factor ends the whole computation

immediately; all factors must be trivial for acceptance. A factor whose scanner hits an object ceiling is unknown, but another factor can still prove knottedness unless the global time budget itself has expired.

6.1 The linear descending test

For a cyclic traversal, let B be the number of crossings first encountered underneath from a chosen basepoint. The diagram is descending exactly when $B = 0$. Moving the basepoint past one visit reverses the first-encounter status of only that crossing. Its contribution changes by $+1$ if the passed visit was over, and by -1 if it was under. Compute B at one start in linear time, then sweep all starts with constant work per visit. Repeating for the opposite orientation gives an $O(n)$ procedure. The baseline recomputes first visits for each possible start, taking $O(n^2)$ time.

A descending diagram is an unknot: after cutting at the basepoint, height can be assigned monotonically in traversal order, and the resulting long arc can be straightened before reconnecting its ends. Failure to find a descending start is not a negative certificate. Our implementation returns the actual successful dart and preserves this one-sided use.

Theorem 6.1. *On a validated classical knot diagram, every completed decision of the new recognizer is mathematically sound, assuming the correctness of the stated invariant theorems and the implemented exact local operations. With no resource ceilings and sufficient memory it terminates with a decision.*

Proof. R1 and R2 moves preserve knot type and strictly decrease crossing number. The descending test only accepts trivial knots. The cut routine produces connected-sum decompositions by Lemma 5.1. The Jones residue only rejects by Proposition 4.1; Alexander rejects only a polynomial different from its normalized unknot value. Neither filter’s failure accepts an unknot.

An unresolved factor is sent to the same finite crossing-complex construction as in the baseline. Every modified cancellation is a chain-homotopy equivalence, and the caches only reuse exactly equal local computations. Its total rank gives the exact decision described in Section 2. Equations (6) and (7) justify combining summand decisions. All loops are finite on an unlimited run, so an unresolved factor cannot be left behind. Resource exceptions instead yield UNKNOWN and are not used as evidence for either mathematical verdict. $\square$

The qualification concerning implemented local operations is not a claim of formal verification. Ordinary source code can have defects; the independent comparisons and algebra tests in Section 9 provide substantial empirical checks, not a machine-checked proof of the Python interpreter or of the whole implementation.

7 Complexity: what is proved and what is not

7.1 An unrestricted exponential upper bound

Let n be the number of crossings in a normalized PD diagram. At one crossing, an old matching has two smoothing choices and each creates at most two new closed circles. Delooping therefore makes at most $2 \cdot 2^2 = 8$ objects from one old object. Before reductions, the number of objects in any stage is bounded by 8^n . Cancellation never increases it.

A Hom-space normal form has $O(n)$ possible circle keys, so a morphism has at most $2^{O(n)}$ square-free terms. Geometry construction, composition, differential storage, and an individual Schur update are polynomial in the explicit object count and morphism-support sizes. There are at most a polynomial number of candidate updates in the explicit stage size: each cancellation removes objects, and each one touches at most the square of the remaining number of objects. The heap can have duplicate candidates, but at most a polynomial number in these same sizes. Consequently these deliberately loose estimates still give

$$T_{\text{scan}}(n) = 2^{O(n)} \quad (8)$$

including polynomial-size label and object-index arithmetic. This is an upper bound, not an empirical model predicting a useful base for the exponential.

The Alexander routine is fraction-free elimination over integer polynomials. Degrees of relevant minors and bit sizes of their coefficients are polynomial in diagram size; polynomial arithmetic and pivoting therefore add polynomial time. Initial canonicalization adds a polynomial in the actual input bit length when external labels have long encodings. The ordinary statement in n assumes normalized labels of $O(\log n)$ bits, as supplied to the backend by the validated constructors.

7.2 The visible-factor parameter

Definition 7.1. Let $m(D)$ be the maximum crossing count of a leaf returned by the implemented visible decomposition after initial simplification. For the already accepted empty diagram, set $m(D) = 0$. This is an algorithmically produced parameter of the input projection, not the crossing number of the largest topological prime factor.

Theorem 7.2. *With default bounded Jones filtering and no terminating resource limit, the recognizer has the bound*

$$T(D) = \text{poly}(n) + n2^{O(m(D))}. \quad (9)$$

The optional factored-rank computation satisfies the same form of bound with m instead defined as its largest raw-input decomposition leaf, since that API does not initially simplify. Integer rank arithmetic and degree convolution add polynomial overhead.

Proof. Simplification, the descending sweeps, and repeated cut searches take polynomial time. There are at most n nonempty leaves. Each has at most $m = m(D)$ crossings, and further reduction cannot increase this number. The bounded Jones filter contributes polynomial overhead; exact Alexander computation is polynomial in its factor size. Bound (8) on every remaining factor gives at most $n2^{O(m)}$ scanning work. Early rejection or memoization can only reduce this bound.

For complete factorized ranks, apply the scanning estimate to the raw decomposition leaves of that API. A raw degree lies between zero and the total crossing count. A count is at most the full cube dimension, hence has $O(n)$ bits by the same exponential dimension bound. Products and successive convolutions of the $O(n)$ -length degree arrays therefore cost polynomial time in n . The large numerical value of a rank does not require an equally large number of storage cells when it is returned as an integer. $\square$

Corollary 7.3. *The delivered recognizer is polynomial-time on inputs with $m = O(\log n)$, and has the requested $n^{O(\log n)}$ bound on inputs with $m = O((\log n)^2)$. In particular, visible connected sums of bounded-size diagrams are polynomial-time inputs.*

Proof. Substitute the respective bounds into (9), using $2^{O((\log n)^2)} = n^{O(\log n)}$. $\square$

The condition on m is an additional hypothesis. There is no argument that all knot diagrams have small m , and no such claim is made in the code or its JSON output. Without that condition, m can equal n .

7.3 Why a thin boundary does not prove fast Khovanov rank

The baseline README suggests that its exponential part depends only on boundary size. That statement overlooks multiplicities. A constant-width scan of repeated bounded-size summands still ends with $2 \cdot 33^k$ objects for the family in Section 5.3. There are only finitely many matching types at bounded boundary size, but arbitrarily many copies of those types in a chain complex. The final boundary is even empty.

A width-only count of matching types therefore cannot bound this implementation’s full homology computation. This is not an objection to local scanning as a practical algorithm. It identifies the exact quantity that must be compressed or avoided: multiplicities and their differentials, not just topological types of frontier states. Visible factorization solves that problem for an exposed tensor-product structure, not for arbitrary prime diagrams.

7.4 Attempting to reach the hierarchy target

Lackenby’s Section 9 analyzes a hierarchy with maximal length L and maximal surface pattern-complexity g . Encoding the complexity sequence as base $g + 1$ digits bounds the number of step executions by

$$L(g + 1)^L. \tag{10}$$

That section explicitly does not provide the required bounds on both parameters or precise costs for all the steps [6]. It cannot simply be read as a proved running time in the crossing number.

Here is the conditional calculation relevant to the requested target. If an implemented hierarchy representation has polynomially bounded g , depth $L = O(\log n)$, polynomial-size intermediate encodings, and polynomial-cost operations, then (10) is $n^{O(\log n)}$. Removing any of these premises leaves a gap. A short list of abstract operations or a lexicographic termination argument alone does not establish the premises.

We examined whether the present scanner could supply this compression by small frontier states. The multiplicity obstruction above defeats that route in general. We instead implemented the concrete special case where the projection exposes a tensor decomposition, obtaining Theorem 7.2. The hierarchy-specific operations in the talk’s final diagram—hierarchical multisurface construction, Cheeger-region use, Haken-lemma transformations, weak reduction, and multisurface simplification—are not implemented here. No executable routine is misrepresented as implementing them, and the optional limits are not substitutes for their cost analysis.

This describes the boundary of the present result. It is neither a refutation of the announced hierarchy approach nor a claim that such an implementation cannot be developed. The delivered improvement is the verified-in-experiments exact scanner and filter system, with the restricted complexity theorem just proved.

8 Reproducible experiments

8.1 Protocol and machine

The included `tools/benchmark.py` creates an isolated Python process for each sample. Both backends are imported before the timer; JSON parsing and construction and validation of the same diagram are also outside the timer. All algorithmic work for the selected operation, including order selection when used, is inside. There is no cache warming between samples. Completed rows have three samples, summarized by their median. A capped row stops after its first unsuccessful sample rather than pretending to have a completion median.

Measurements were collected with Python 3.13.5, Linux x86-64 with glibc 2.41, on a virtualized host identifying its processor as AMD EPYC 9V74. The supplied baseline runs in that same Python and environment. The cooperative budget is 20 seconds; the parent process also has a 24-second emergency timeout. All recorded capped rows in this run reached the cooperative limit. Neither method is forced to a smaller object ceiling. The routine full benchmark does not enable the expensive $d^2 = 0$ debug checks; correctness runs enable them separately.

There are 37 backend/case/operation rows and 105 individual samples. Raw JSON includes the full answers, orders, work counters, timing, and process peak resident memory where available. These numbers are microbenchmarks of the included fixtures, not estimates of performance on a random distribution of knots. The benchmark is single-threaded and no external algebra package or compiled accelerator is used.

8.2 Default end-to-end algorithmic timings

Fixture	Baseline (ms)	New (ms)	Ratio
Conway, 11 crossings	35.630	0.915	38.9
Kinoshita–Terasaka, 11	39.297	0.886	44.3
Hard unknot, 8	1.621	1.819	0.89
Five-strand example, 36	8.082	5.217	1.55
Unknot braid, 40	6.411	6.353	1.01
Torus knot $T(3, 5)$, 10	0.590	0.904	0.65

Table 1: Default recognition. The ratio is baseline time divided by new time; a value below one is a regression. Inputs and imports are excluded for both.

Conway and Kinoshita–Terasaka both have Alexander polynomial one in these fixtures, so the old pipeline reaches Khovanov homology. The new modular Jones filter rejects them immediately. This is the source of the roughly 39-fold and 44-fold end-to-end improvements, not just a constant-factor scanner optimization.

The new pipeline is *not* universally faster. Extra exact filters add overhead to the small hard unknot, which still needs positive recognition, and to $T(3, 5)$, which the old Alexander test already rejects cheaply. The latter becomes approximately 1.53 times slower, although still submillisecond in these measurements. Users with such a workload can disable Jones and/or visible factorization. No general dominance claim is justified by the data.

8.3 Raw scanning, without preprocessing or invariant shortcuts

Fixture	Baseline (s)	New (s)	Reduced rank
Conway	0.033667	0.010819	33
Kinoshita–Terasaka	0.039130	0.011889	33
Hard unknot, 8	0.002139	0.001898	1
Five-strand example, 36	> 20 (capped)	1.284672	2949
Unknot braid, 40	0.008588	0.004003	1
Torus knot $T(3, 5)$	0.010973	0.010467	7

Table 2: The same original PD is sent straight to each scanner. The capped entry is not a measured runtime; the difficult-case speed ratio is greater than 15.5 relative to the 20-second budget.

The hard 36-crossing case already appears in the supplied benchmark data. That older file records a 600-second raw-scan timeout under Windows and Python 3.14.4. We do *not* divide that time by a new Linux measurement. The comparison in Table 2 reruns the actual original code on the same host, with a clearly reported 20-second cap.

For this diagram, the reduced rank 2949 is new computational output relative to that capped baseline row. The raw result has unreduced rank 5898 and maximum boundary size eight. These figures illustrate both the practical value and the limitation of narrow scans: a thin scan can still have substantial homology multiplicities.

The default old pipeline already decides this example in approximately 8 milliseconds after six R2 moves reduce it to 24 crossings and Alexander rejects it. The new pipeline uses a Jones residue of 426151971 modulo 1000000007, obtained with 14 maximum states and 400 transitions. Its total time is approximately 5 milliseconds. Thus the large raw-scanner improvement must not be presented as a comparable end-to-end improvement on this input.

8.4 Ablating pivot policy and algebra caching

36-crossing raw-scan variant	Seconds	Reduced rank
Original algebra, original stack policy	> 20	not completed
New caching, original stack policy	> 20	not completed
Original uncached algebra, fill policy only	7.627084	2949
New caching and fill policy	1.284672	2949

Table 3: Ablation results. The fill-only worker installs the new elimination loop in the baseline module namespace, retaining the original composition, inversion, crossing maps, and scan-order implementation.

The comparison shows that caching alone does not resolve this particular bottleneck: its stack-policy variant also reaches the cap. Changing pivot selection makes the original uncached algebra finish. Adding the cache and unit shortcuts then improves that fill-only time by approximately 5.94. The four rows do not isolate every small optimization individually, so this is not a claim to have attributed a precise fraction of the speedup to each cache.

The fill-only variant and the optimized variant agree on every reported homological degree count, not just the final total. A separate fill-only run with $d^2 = 0$ checks also completes and

gives rank 2949; its result is saved in `results/hard36-original-algebra-check.json`. This tests the optimized algebra against the original one, but both variants still share the same underlying categorical model. It is not an independent formal proof of that model.

For context, the first successful optimized sample has process peak RSS 129708 KiB, whereas the first fill-only sample has 168744 KiB. These include interpreter and process memory, not just live chain-complex storage. They are not a proven memory bound and should not be extrapolated to arbitrary knots or scan widths.

8.5 Factorization and ordering experiments

Operation	Baseline (s)	New (s)	Ratio
Two Conway summands, forced scan	1.427456	0.010935	130.5
Three Conway summands, forced scan	> 20	0.012196	> 1639
Ordering only, chain of 64	0.014000	0.001749	8.0
Ordering only, chain of 256	0.217324	0.007209	30.1
Ordering only, chain of 1024	3.414980	0.032857	103.9

Table 4: The forced-scan recognition rows disable Alexander in both versions and Jones in the new version, retaining ordinary reductions and the new factorization. Ordering rows time only crossing-order selection.

The two connected-sum rows demonstrate a structural shortcut independently of the Jones filter. The old code calculates a product-sized homology, whereas the new code rejects after the first summand. The twelve-summand factored-rank run takes 0.031764 seconds and computes two distinct normalized PD leaves before reusing them; there are twelve leaves altogether. It returns the full degree convolution as well as 33^{12} .

The ordering examples are the closures of $\sigma_1\sigma_2\cdots\sigma_n$ on $n+1$ strands. Both routines return the same chosen order. The increasing ratios are consistent with the proven change from quadratic to near-linear ordering work. They say nothing by themselves about full recognition time on a 1024-crossing arbitrary knot.

9 Validation and limitations of the evidence

The delivered suite contains the original 19 tests and 17 new tests, all passing. The inherited method-name tests explicitly disable the two new filters so that they continue to exercise the old pipeline path. This is documented in the test source; no failing old assertion was silently deleted. The final recorded run with `PYTHONHASHSEED=0` reports 36 tests passing in 13.120 seconds.

The new tests are mostly property loops rather than single examples. They compare the heap ordering with the original ordering for every start on 100 diagrams; compare the linear descending predicate with the original one on 200; compare both pivot policies with an independent dense reduced cube-of-resolutions implementation on 100; and compare the default recognizer with that dense reference on another deterministic set of 120 generated diagrams. Small braid closures are restricted to one component by a separate permutation test.

For the scalar filter, 160 small diagrams are checked against an independent 2^n state sum at two values of A , with varied crossing orders. Other tests check that known unknots are never rejected,

that legal row rotations preserve writhe and invariants, and that bad ring specializations, malformed scan orders, non-finite time budgets, and undersized work caps are handled correctly. A composite admissible modulus is tested, so correctness does not tacitly depend on primality.

Algebra tests compare memoized composition with the retained original routine and test mutable-alias protection. All 128 three-arc units are verified to square to the identity. Connected-sum tests compare raw scanning, factored degree convolution, and the independent dense reference on nine pairs of small diagrams; cut replay is checked and a tampered cut is rejected. A twelve-Conway rank test checks exact large-integer output. The difficult 36-crossing scan is run with $d^2 = 0$ checking enabled.

In addition to the suite, the independent dense reference computes reduced rank 33 for each of Conway and Kinoshita–Terasaka, in approximately 0.23 and 0.20 seconds. This cross-check is feasible despite their awkwardness for the old filter pipeline: eleven crossings are small enough for a reference cube. Its raw outputs, Alexander values, and reducing-move counts are preserved.

What this does not establish. There is no Lean formalization of these changes, no proof of correctness of the Python runtime, and no exhaustive validation over all diagrams. The 36-crossing rank was checked by original versus optimized local algebra and by chain-complex consistency, not by an independent dense enumeration of its 2^{36} states. Property tests share the inherited PD parser and, in some cases, its traversal routines. Numerical agreement therefore does not remove all correlated-error risks. These limits are stated so that the test count is not mistaken for a proof certificate.

10 Interfaces, evidence, resource budgets, and reproducibility

10.1 Commands and library entry points

The package runs directly from the extracted deliverable:

```
python -m fastunknot recognize examples/conway.json
python -m fastunknot recognize examples/hard_unknot_8.json
python -m fastunknot khovanov examples/random5_36.json --check-d2
python -m fastunknot khovanov examples/conway_sum_12.json --factor
python -m unittest discover -s tests -v
```

The third command intentionally bypasses simplification and invariant tests. The fourth explicitly opts into factored ranks. Keeping both interfaces prevents confusion between a faster full-rank scanner and a shortcut for recognition or rank multiplication.

The public constructors accept PD, braid, and the inherited rectangular-grid formats. For example:

```
from fastunknot import Diagram, recognize
D = Diagram.from_braid(3, [1, -2, 1, -2])
answer = recognize(D, seconds=30, max_objects=200_000)
print(answer.status)
print(answer.to_json())
```

The raw `khovanov_rank(pd)` function assumes an already validated PD; normal use goes through `Diagram.from_pd` or another checked constructor. The dataclass itself must not be used to bypass

those checks.

The old recognition options remain available. New ones are `use_jones`, `use_factorization`, the two Jones work caps, and `pivot_strategy`. The corresponding CLI flags include `-no-jones`, `-no-factor`, and `-pivot lifo`. A completed mathematical decision has exit code zero, a resource-limited unknown has code three, and invalid input or a rejected witness has code two.

10.2 Evidence is contextual, not a universal certificate

A standalone Jones result can be reproduced on the same diagram:

```
python -m fastunknot jones examples/conway.json > witness.json
python -m fastunknot verify-jones examples/conway.json witness.json
```

The verifier recomputes the finite-ring evaluation and compares the relevant fields; it does not trust the claimed residue. This is a reproducibility checker, not a succinct polynomial-time checker for a short certificate. On adversarial inputs it needs external process limits.

The cut-tree replay function reconstructs the leaf diagrams from explicit two-edge-cut records. A full pipeline result has additional context: initial R1/R2 traces precede the cuts, and each factor can have its own reduction trace. Thus a leaf's Jones witness refers to its *post-reduction leaf*, not necessarily to the original input PD. Those reductions must be replayed before the respective witness is checked. The current package does not offer a complete independently implemented verifier for every pipeline result. Scanner rank output is computational evidence, not a spanning disk or a formally checked homology certificate.

10.3 Limits are explicit but cooperative

`max_objects` constrains only the scanner's planned delooped object count. An exact Jones or Alexander rejection is still valid even if that ceiling is tiny. A Jones-specific state or transition cap skips Jones and falls back; exhaustion of the shared time budget stops with UNKNOWN. Invalid negative or non-finite budgets are rejected instead of being used as hidden unlimited modes.

Deadline checks are inserted in reduction, descent, factorization, polynomial elimination, state transitions, object construction, and cancellation loops. They are not a hard interruption mechanism: a single large Python arithmetic or composition operation, or a debug d^2 check, may run past its deadline. Similarly, caches are bounded in entry count but not total bytes, and the scanner ceiling does not bound morphism support sizes or Python overhead. Applications requiring strict wall-clock or memory isolation should run the process with operating-system limits. `MemoryError` in the high-level recognizer becomes UNKNOWN, but graceful recovery cannot be guaranteed after an external out-of-memory kill.

10.4 Rebuilding the measurements and article

The unchanged comparison code, worker, example generator, and raw results are included. Run

```
python tools/benchmark.py --repeats 3 --cap 20
python tools/summarize_results.py
```

to regenerate the measurements and readable summary. The first command takes several minutes because it intentionally waits on capped baseline cases. `benchmark_worker.py` exposes the individual

operation and ablation variants for targeted reruns. Fresh processes include newly empty caches in both variants, and input construction remains outside both timers.

The article has no external figure or bibliography dependencies. From its `docs/` directory, two runs of

```
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

rebuild its PDF and cross-references. LaTeX is not needed to use the software. The original license is retained, and third-party papers and slides are cited rather than redistributed in this ZIP.

11 Conclusion and the next concrete bottleneck

The delivered improvement is a working exact recognizer, not an unimplemented proposal. It changes the scanner’s elimination behavior enough to complete a formerly capped raw scan, adds an independent bounded rejection test that helps the two eleven-crossing Alexander-trivial fixtures, and avoids an exponential tensor-product basis on exposed connected sums. The strongest proved bound is (9), not an unrestricted quasi-polynomial bound.

For difficult diagrams with no exposed factors and with filters that return one, the remaining bottleneck is a genuinely large categorical chain complex. A next implementation step would be to represent repeated boundary matching multiplicities by blocks and seek algebraically certified low-rank factorizations of the differential blocks. Such a representation must preserve the Schur updates and account for the cost of operations on those blocks; merely storing multiplicities as integers is insufficient when nontrivial differentials couple the copies. For the separate hierarchy approach, a compressed topological state and costed implementations of its moves are still required. Neither direction is smuggled into the present complexity claim.

References

- [1] D. Bar-Natan, *Fast Khovanov homology computations*, Journal of Knot Theory and Its Ramifications **16** (2007), 243–255. arXiv:math/0606318. Author’s publication page: FastKh.
- [2] P. B. Kronheimer and T. S. Mrowka, *Khovanov homology is an unknot-detector*, Publications Mathématiques de l’IHÉS **113** (2011), 97–208. arXiv:1005.4346.
- [3] L. H. Kauffman, *State models and the Jones polynomial*, Topology **26** (1987), 395–407. doi:10.1016/0040-9383(87)90009-7.
- [4] M. Lackenby, *Unknot recognition in quasi-polynomial time*, February 2021, 109-page talk slides. The exact version supplied in docs/quasipolynomial-talk.pdf. Author-hosted February slides. Physical pages 13, 60–74, and 109 are relevant here.
- [5] M. Lackenby, *Unknot recognition in quasi-polynomial time*, March 2021 Oxford slide variant, 109 pages. Author-hosted Oxford variant. Cited only to distinguish its stated exponent from the submitted February file.
- [6] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, 25 July 2026, version 1, 54 pages. arXiv:2607.23350v1. In particular Section 9, physical pages 34–35. Public version checked on 18 September 2026.
- [7] Knots contributors, *fastunknot: exact unknot recognition with a scanning Khovanov backend*, user-supplied Knots.zip, fast/ directory, September 2026. The actual comparison code is retained in baseline/fastunknot/; source provenance and the original license accompany this report.